The Codebreaker & The Safety Net: A Team Challenge

Challenge Overview: Your team has intercepted an encrypted, corrupted transmission from an old satellite. The message is trapped inside a list of raw data. Your job is to decode the message, clean it up, and build a "bulletproof" script that handles any system failures gracefully.

Part 1: The Challenge Setup (Copy this Code)

This is the starter code. It contains the data they need to process and some intentional landmines that will cause exceptions.

Python

```
None
# The corrupted satellite data feed
satellite_feed = [
    "72-69-76-76-79",       # Encrypted Word 1 (ASCII decimal
codes)
    "87-79-82-76-68",       # Encrypted Word 2 (ASCII decimal
codes)
    42,                     # Global positioning float code
(Corrupted data!)
    "  _sYstEm_oNlInE_  ",  # System status string
    "0"                     # Battery critical override multiplier
]
```

Part 2: The Team Mission Tasks

Task 1: Characters & Strings vs. Computers

The first two items in `satellite_feed` are strings containing numbers separated by dashes. These numbers are **ASCII decimal codes**.

- Write a function that takes `"72-69-76-76-79"` and `"87-79-82-76-68"`, splits them by the dashes, converts the numbers into actual characters, and glues them into words.

- *Hint:* Look into Python's built-in `chr()` function, which turns an ASCII number into a character!

Task 2: Strings in Action (The Cleanup)

The 4th item in the feed is `"  _sYstEm_oNlInE_  "`. It's messy. Use string sequencing, slicing, and string methods to:

1. Strip away the whitespace at the beginning and end.
2. Remove the underscores.
3. Convert the entire phrase into **LOWERCASE**.
4. Print the final cleaned string.

Task 3: The Anatomy of Exceptions (The Crash Test)

Without adding any error handling yet, write code that tries to loop through the *entire* `satellite_feed` list and run your Task 1 decoding function on every item.

- Watch the program crash when it hits the 3rd item (`42`).
- Look at the final line of the crash message (the traceback). What is the **exact name** of the Exception Python threw? Why did it throw it based on what you know about strings?

Task 4: Useful Exceptions (The Bulletproof Shield)

Now, refactor your loop using a `try...except` block.

- Catch the specific exception you identified in Task 3 so the program doesn't crash when it hits `42`. Instead, print a helpful warning like: `"[SYSTEM WARNING]: Skipped corrupted non-string data."`
- Inside the same loop, try dividing the number `100` by the 5th item in the list (`"0"`). This will trigger another exception. Catch this *specific* new exception as well, and print: `"[SYSTEM WARNING]: Cannot divide by zero override."`